

カルネージハートシリーズのAIプログラム仕様調査

カルネージハートのOS設計概要

カルネージハート（初代～PSP版）では、プレイヤーが機体（OKE）の**行動プログラム**を専用の「チップ」を使って組み立てます。プログラムはテキストではなくフローチャート形式で、ボード上にチップを配置し線で繋いで作成します^①。プログラムの開始地点（ボード左上のSTART）からチップを順にたどり、最後はボード端のRETURNに達すると自動的にまたSTARTに戻ってループ実行されます^②。各チップには固有の機能があり、条件判断用チップでは**Yes/Noの分岐**が発生し、動作系チップでは指定のアクションを実行します^③。チップは色分けされカテゴリごとに分類されており、たとえば**赤いチップ**はセンサー/条件判断、**青いチップ**は移動や攻撃などの行動、**緑のチップ**はターゲットロックや自動旋回・オプション使用などの指示系、**黄のチップ**はウェイト（待機）など特殊な制御用途、といった具合です（シリーズや作品によって若干異なる）^④^⑤。以下、このプログラミングシステムの要点を項目別に詳しく説明します。

1. 条件分岐に使われるチップと構文

条件分岐（分岐処理）は**専用の条件チップ**によって実現されます。これらのチップは「○○か？Yesなら～ / Noなら～」という**if文相当の判定**を担い、Yes出力とNo出力の2方向の信号（矢印）を持ちます^⑥^⑦。プログラムは条件チップをボード上に置き、Yes側・No側それぞれに次に実行すべきチップを繋げます。たとえば「敵との距離100m以内か？」という条件チップでは、Yes分岐に回避行動のチップを繋ぎ、No分岐には次の判定へ進むよう別のチップを繋ぐ、といった具合です^⑧。このように各条件チップがプログラム内で**if文ブロック**の役割を果たし、チップの組み合わせ次第で複雑な条件式を表現できます。なおプログラム上ではインデントやブロック構造は線で表現されるため、**入れ子（ネスト）**の条件分岐もチップを繋ぐ順序と配置によって実現します^⑨^⑩。

条件チップの構文自体は基本的に真偽（二択）判定ですが、**多段階の比較**にも対応できます。各条件チップにはパラメータ設定があり、閾値や比較演算子を指定可能です。例えば「残弾数が ≤ 10 発か？」「機体温度が $\geq 85\%$ か？」「タイマーが $= 5$ 秒経過したか？」といった具体的な条件設定ができます^⑪。判定の結果Yes/Noという**2出口**を持つ点は共通で、直接一度に多岐分岐（3分岐以上）するチップは存在しません。しかし**複数の条件チップを組み合わせる**ことで事実上の多分岐を表現できます（後述するように、if-elif相当の連続判定や4通りのケース分け等も可能です）^⑫。全体として、カルネージハートの条件分岐は「**Yes/Noの二択を積み重ねて複雑な分岐論理を作る**」構造になっています。

2. 条件として使用できるセンサーや変数類

条件チップ（センサーチップ）の種類は非常に豊富で、機体の周囲状況や内部状態、乱数やカウンタ値まで様々なものを判定できます。主なセンサー/変数と用途は以下のとおりです。

- ・**距離・方角センサー**（機体探索系）：周囲の敵・味方機を探知します。一定範囲内（距離m・角度°を指定可能）に敵機が存在するかどうか、特定方向に障害物があるか、飛来する弾（ミサイル等）があるか、などを検知してYes/Noを返します^⑬^⑭。例えば「前方360°800m以内に敵がいる？」^⑮、「極座標半径70m以内に高速飛翔体（敵弾）が飛んできている？」^⑯といった条件で分岐可能です。※ターゲットロックしている敵とは別の「未確認機体」をレーダー外で発見するような高度な探知も可能です^⑰。障害物探知では**戦闘エリアの境界**も判定でき、例えば「右側が戦闘エリア外か？」という条件もあります^⑱。

- ・**機体状態センサー**（状態判断系）：自機の内部状態をチェックします。具体的には**耐久力（HP）**の残量や**機体温度（熱量）**、弾薬残数などを閾値付きで判定できます¹⁷⁶。例えば「HPが50%未満か?」「機体温度が80%以上か?」等の条件でYes/Noを出力し、必要に応じて**修復装置や冷却装置などオプションを起動**する処理に繋がります¹⁸。この種のチップは**オプション装備の自動発動**（自動修復や自動冷却）のトリガーによく使われます¹⁸。
- ・**ターゲット情報センサー**（ターゲット判断系）：現在ロックしている敵ターゲットに関する情報を判定します。例として**ターゲットとの距離や方角**（自機から見た敵の位置関係）¹⁹、**ターゲットの動作状態**（相手が移動中か攻撃中か等）²⁰、**ターゲットの機体コード**（機種識別）²¹、**自機が敵にロックオンされているか**²²、**敵の武装タイプ**（相手がどの武器を持つか）²³など細かな条件判定が可能です。これらは敵ごとに戦術を変えたり警戒動作を取る際に有用です。
- ・**フラグ・カウンタ**：プレイヤーが任意に設定し得る**変数**も存在します。代表的なのは**カウンタ**（COUNTER）で、自由に数値（1～99）を保持・加減算できます²⁴。カウンタは初期値0からスタートし、プラス/マイナスや値セットのチップで操作しつつ、その値が一定以上/以下になったかを**カウンタ判断チップ**で分岐できます。例えば「カウンタAが5になったか? YESなら特定行動へ」などと使い、ループの回数制限や時間経過の簡易カウントに応用できます。また**通信フラグ（チャンネル）**もあり、同じ部隊のOKE間で**5色の信号を共有**できます²⁵。「チャンネル赤に信号が来たか?」といった条件で味方との連携行動を取ることも可能です（味方機でチャンネルに色信号を送信するチップがあり、他の機体がそれを受信判断する）³。
- ・**ランダム**：**乱数分岐チップ**も用意されています²⁶。任意の確率（%）を設定してYesを出すチャンスを調整でき、例えば「50%の確率で左ジャンプ、50%で右ジャンプ」といった**ランダムな行動選択**を組み込めます³。このランダム要素により毎回ワンパターンにならない動きを実現できます²⁶。
- ・**時間**：**時間判断チップ**では戦闘開始からの経過時間や残り時間を条件にできます²⁷。「戦闘開始5秒経過したか?」や「残り時間〇秒以下か?」といった判定が可能です、時間によって戦術を切り替えるAIを組むこともできます。
- ・**手動入力**：一部には**ボタン判定チップ**も存在し、プレイヤーの手動操作ボタンが押されたかを条件にできます²⁸。（※通常はOKEは自律行動しますが、手動操作モード時や特殊なケースで使用）。

以上のように、**敵・味方の周囲環境から自機のステータス、擬似乱数やカウンタ変数**まであらゆる情報を条件分岐に利用できます。組み合わせ次第で「敵が近くにいて弾が飛んできたら回避」「一定時間経過後に攻撃モード移行」「味方が特定信号を出したら連携攻撃」等、細かなAI挙動を設計できます。

3. 分岐の種類（True/Falseの二択か、多分岐・数値比較の可否）

基本は**True/Falseの二択分岐のみ**ですが、前述のとおりパラメータ設定により数値比較を行うチップもあります。各条件チップでは適用する比較演算が選択可能で、閾値に対して「以上/以下」や「等しい」などを指定して判定できます²⁹。例えば状態判断チップでは「耐久力が〇%以下ならYes」「温度が〇以上ならYes」など細かな条件設定が可能です。これらはいずれもYesかNo（満たす/満たさない）の二者択一の結果を返し、チップ自体が直接3分岐以上を持つことはありません。

しかし、**多分岐のロジック自体は実現可能**です。複数の条件チップを**段階的に連結**することで、いわゆるif～elif～elseに相当する処理やスイッチ文に近い分岐が組めます。例えば2つの条件A・Bから4通りのケース（「AもBも成立」「Aのみ成立」「Bのみ成立」「どちらも不成立」）を区別したい場合、A判定→そのYes枝内でB判定→それぞれのYes/No枝を整理することで4分岐を表現できます¹¹。同様に、条件チップを直列や並列に繋ぐことで**AND条件**（両方Yesの場合に次へ）や**OR条件**（どちらかYesなら次へ）、**NOT条件**（条件の

否定)も表現できます⁹³⁰。このように単体チップでは二択でも、組み合わせ方次第で**複合条件**や**多岐分岐**を自由に設計できます。

数値の大小比較については、上記のように**しきい値付き条件チップ**(状態判断やタイマー等)で対応します。たとえばカウンタ値やタイマー値のチェックは「>= ○ならYes」等と設定でき、疑似的に「>」「<」「=」などの比較を行えます²⁹。一方、特定の値ごとに分岐先を変える**スイッチ文的な多分岐**(例:「変数Xが0なら処理A、1なら処理B、2なら処理C」)は、残念ながら1つのチップでは実装できません。その場合も「まずX==0?で判定→Noなら次にX==1?→…」と順に条件を重ねて対処します。このように**全ての分岐は二択の積み重ね**ですが、工夫次第で多様な条件分岐ロジックを組み立てることが可能です。

4. 実行モデル (チップ処理サイクル、優先順位、実行結果の扱い)

プログラムの実行モデルは基本的に「ループ処理」です。各OKEは戦闘中、自身に設定されたチッププログラムを常に**最初 (START) から順番に実行**し、どの経路を通っても最後にはRETURNによってまた冒頭に戻ります²。このループはハードに搭載したCPUの性能によってスピードが異なり、上位CPUほど一連の処理を短時間で高速反復できます。例えば中型CPU「MP-100L」は実行速度90に対し上位の「MP-100H」は240と約2.6倍の速度を持ち、プログラムサイクルが速い(リアルタイムで反応が鋭い)ことを示します³¹³²。CPUはまた記憶容量(配置できるチップ数)も制限があり、複雑なAIほど大容量・高性能のCPUが必要になります³³。

条件分岐の優先順位については、基本的にプログラム上で**先に書かれている (STARTに近い) 判定ほど高頻度でチェックされます**²。毎ループ必ずSTART直後から実行するため、冒頭に置いた処理はループ毎に必ず通ります。一方、末尾近くにある処理は途中の分岐次第ではスキップされることもあります。したがって「**重要な判断ほどプログラムの早い位置に置く**」ことが推奨されています³⁴。実際の例では、最優先で使いたいオプション(自動修復や緊急回避など)の判定を開始直後に配置し、その後で移動や攻撃といった処理に分岐していく構成が基本です²¹⁸。

なお、**動作チップ (行動チップ) の実行優先度**も一部存在します。公式攻略ガイドによれば、OKEの行動には「**ジャンプ系 > 攻撃系 > 移動系**」の優先順位があります³⁵。例えば同じループ内でジャンプと射撃の両方が命令された場合、ジャンプ動作が最優先され、次に攻撃、最後に移動という形で処理されます。この仕組みにより、衝突しうる命令が同時に出了場合でも重要度の高い動作がキャンセルされず実行されます(ジャンプ中は移動命令を無視する等)。もっとも、プログラム設計時には通常これらが競合しないよう**適切な条件分岐で制御**することが望ましいです。

動作チップの「通過/停止」モードについても理解が必要です。各アクションには**同時実行性**に関するモード設定があり、チップごとに「**通過 (パス) モード**」か「**思考停止 (ホールド) モード**」を選べます。通過モードで実行した動作は**後続のチップ実行を妨げず**(プログラム処理を止めず)に並行実行されますが、停止モードの動作は**その完了までプログラムを待機**させる効果があります³⁶。初期の作品ではほとんど全ての行動が次の命令で上書きキャンセルされてしまい(=実質的に直列実行のみ)、移動しながら射撃する等の同時動作には工夫が要りました³⁷。しかしPSP版(エクサ)ではこのモード選択が可能となり、**射撃を通過モードで指示した後に移動/ジャンプを実行すると移動しながら射撃が可能**になるなど大幅に自由度が増しています³⁸。逆に移動を通過モードで行った後に射撃を実行すれば「走り撃ち」も可能です³⁸。この違いにより、たとえば回避判定の後に射撃判定を繋いでも両方機能させることができますし、移動中に次の行動判定へ移る高速なAIループも作れます³⁶。ただし停止モードで発動した動作は次のループまで他命令を受け付けられないため、意図しない箇所プログラムが「固まる」ことのないよう注意が必要です。

結果の扱いという観点では、基本的に各チップの実行は即座に結果を反映します。条件チップはYes/Noの判定結果に応じてプログラムの流れ(次に実行するチップ)を即座に決定しますし、動作チップは実行された瞬間から機体の動きや状態に影響を与えます³。例えば「武器発射」チップに到達すればその瞬間に射撃動作が行われ、以降のチップ実行に移ります。また一つのループ内で**複数の動作チップが順に実行**されるケー

スもあります。前述の通過モードを活用すれば、**同じフレームで移動しつつ攻撃する**ような並行動作も可能です³⁸。ただし上位優先の動作がある場合、下位の動作はキャンセルまたは遅延する可能性があります（例：ジャンプ中は射撃動作が完了まで保留されるなど）。このあたりの詳細挙動は作品や状況によりますが、総じて「**プログラムで指示された行動はループ内で順次処理され、機体には可能な範囲で即適用される**」と考えてよいでしょう。

5. 条件チップや行動チップの具体例

ここでは実際の**条件チップ**と**行動チップ**の使用例をいくつか挙げます。

- **オプション装備の自動使用**: 自機の状態に応じて修復装置や冷却装置などを自動発動するプログラム例です。¹⁸に示すように、「HPが減っている？」→Yesなら**オプション1（修復装置）使用**→次の判定へ、「機体温度が上がり過ぎ？」→Yesなら**オプション2（冷却装置）使用**→次へ、「ミサイルが飛来している？」→Yesなら**オプション3（誘導妨害装置）使用**→次の処理へ…という形でチップを連結します¹⁸。この例では各状態センサー（耐久/熱/被ロックオン）を条件チップでチェックし、Yesの場合それぞれ対応するオプション使用の**行動チップ**（緑色チップ）を実行しています。Noの場合は次の条件へ順送りし、いずれも該当しなければ何もせず次の大きな処理（回避判断など）へ移ります。結果として、機体が損傷したら即修復し、過熱したら冷却し、ミサイルを捕捉したら自動でジャミングする、といった**自己防御ルーチン**が完成します。このような**状態判断系チップ+オプション動作チップ**の組み合わせは、耐久力管理やオーバーヒート対策の基本形です。
- **近接警戒と回避行動**: 敵との距離や飛んでくる弾を感知して回避する例です。まず「敵が100m以内にいるか？」を判定し、Yesなら即座に**後方ジャンプ**（その場で後ろに大きく飛ぶ青チップ）を実行します⁸。No（敵が遠距離なら）場合、次に「高速飛翔体が接近中か？（敵の射撃が飛んできているか？）」を判定します¹⁵。これがYesなら回避行動に移りますが、ただ闇雲に避けるのではなく更に細かい条件分岐で**左右どちらに回避するか**を決めています。¹⁵の例では「（右に回避して）**右側が戦闘エリア外ではないか？**」「（さらに）**右150m以内に味方がいないか？**」を順にチェックし、どちらも問題なければ**右ジャンプ**、どちらか一つでもYes（右に障害がある場合）は**左ジャンプ**する、としています¹⁵。このように複数条件を組み合わせ、**安全な方向を判断して回避**する高度な処理も可能です。



上図はカルネージハートEXAにおけるOSプログラムエディタ画面の一例です。赤い条件チップ（左側）と青い行動チップ（右側）が矢印で繋がれており、フローチャート形式で回避AIが構築されています。このプログラムではまず「ターゲット機体が指定範囲内に居るか？」を赤チップで判定し、Yesなら青チップの「後方

ジャンプ」動作に繋がっています（画面中央の垂直方向の接続）⁸。Noの場合は右方向の矢印に沿って次の判定「高速飛翔体が半径70m以内に存在するか？」へ進みます¹⁵。この条件がYes（敵弾接近中）の場合、さらに分岐して「右側が戦闘エリア外か？」（赤チップ）と「右150mに味方が居るか？」（赤チップ）の二つをチェックし、どちらもNoなら**右ジャンプ**（青チップ）、いずれかYesなら**左ジャンプ**（青チップ）を実行する流れです¹⁵。このように**複数の条件チップを段階的に繋げて分岐先を選択**している点に注目してください。例えば敵が近くにいれば即後退ジャンプで距離を取り、遠くても弾が飛んできた場合は味方やエリア外を避けつつ左右どちらかにジャンプする、という細かな挙動を一連の条件分岐で実現しています。

・**攻撃動作と発射制御**: 攻撃時にも条件分岐を入れて無駄撃ちや誤射を防ぐ例です。³⁹では射撃処理の直前に2つの条件チェックを挿入しています。まず「射線上に味方機がいるか？」を判定し、Yesの場合は**射撃停止**コマンド（攻撃中止の行動チップ）へ直行します⁴⁰。Noの場合次に「機体熱量が一定以上か？」（オーバーヒート気味か）を判定し、Yesなら射撃せず即RETURN（何もしないで終了）しています³⁹。両方Noで問題なければ、青チップで**ガンサイト起動**（自動照準開始）→**武装1射撃**（狙撃モードで2発発射）→RETURNという一連の攻撃動作に繋がります³⁹。この例では**フレンドリーファイア防止と過熱時の射撃中止**という2つの条件チップ（赤）が、実際の射撃コマンド（青）の前段に配置されています。条件を満たさない場合のみ安全に射撃を実行し、そうでなければ射撃行動をスキップまたは即停止する仕組みです。実際の戦闘ではこのような判定を入れることで、無駄弾を撃ったり味方を誤射するリスクを減らせます。また使用武器や距離によっては「敵が近すぎる場合は射撃ではなく格闘に切り替える」といった分岐を入れることも有効でしょう⁴¹。

・**カウンタと待機の利用**: カウンタ変数や待機チップも具体例を挙げます。**カウンタ**は例えば「ミサイル斉射を○回行ったらリロードに移る」ような使い方ができます。射撃命令と並行してカウンタを+1し、カウンタ値が一定に達したかを条件チップで判定、Yesなら次の行動（弾倉交換動作や別武器への切替など）に分岐させる、といった制御です。カウンタは使い方次第で**ループ回数の制限や時間遅延の実現**に役立ちます。また**待機チップ（ウェイト）**は指定した時間だけ思考を停止する（次のチップ実行を保留する）ものです⁴²。例えば「敵に見つからないよう数秒待ってから攻撃を開始する」場合や、「ジャンプ着地まで他の行動を待つ」場合などに使われます。待機時間中はプログラムの他の処理を進めないため、**無駄なループを回さずに時間経過を待つ**用途に便利です。ただし待機中は他の条件チェックも止まるため、長時間の停止を入れると環境変化に対応できなくなる点に留意が必要です。

以上のように、条件チップと行動チップを組み合わせることで多彩な例を作れます。オプション使用や回避・攻撃・待機タイミング調整・カウンタ管理など、**ゲーム内のあらゆる状況に対処するチップ例**が用意されています。攻略本やユーザー作成のガイドには「OSチップの具体的な接続例」や**フローチャート図**が多数掲載されており、先述のようなパターン以外にも目的別のプログラム例（例えば「援護射撃型」「格闘特化型」「ミサイル回避専念型」など）が紹介されています。そうした例も参考に、自分の狙う戦術に合った条件分岐と行動シーケンスを構築できます。

6. チップのカテゴリと主な種類

カルネージハートのプログラム用チップは、その役割ごとに**カテゴリ分類**されています。大きく分けて「思考系」「条件（センサー）系」「動作系」「指示系」などがあり、色や形のアイコンで判別できます⁴⁵。ここでは代表的なカテゴリとチップの種類を列举します。

・**移動系チップ（動作）**: 機体の移動動作を指示するチップです。例えば**前進・後退・旋回（ターン）**といった基本移動や、二脚/多脚用の**ジャンプ**、車両型用の**急停止**や**旋回停止**、飛行型用の**上昇（Ascend）/下降（Descend）**などがあります⁴³⁴⁴。移動系チップは青色の矢印アイコンで描かれ、信号は1出力（直線的に次のチップへ繋がる）です⁴⁵。なお**伏せ（ダック）**動作など特殊回避も移動系に含まれます⁴⁶。これらは基本的に1回の命令で一定距離/角度だけ動作し、必要に応じて繰り返し呼び出すことで連続的な移動を実現します。

- ・攻撃系チップ（動作）**：武器の使用や攻撃行動を指示するチップです。代表例は**主武装射撃・副武装射撃・格闘攻撃・自爆**などがあります^{47 48}。射撃チップでは射角や射程をパラメータ指定でき（例：前方〇度内を狙う**スナイブモード**等）、副武装は一度に何発撃つか設定することも可能です⁴⁹。格闘は近距離で自動発動的に相手に組み付く動作です⁵⁰。**自爆**は自機を爆発させ周囲に大ダメージを与える最終手段のチップです⁵¹。攻撃系チップも基本1出力信号で直後の処理に繋がります。攻撃は移動との並行実行が可能なため（前述の通過モード参照）、**移動しながら射撃**といった組み合わせもプログラムできます³⁸。
- ・オプション系チップ（指示）**：機体に搭載したオプション装備を使用するチップです。例えば**誘導妨害装置（ミサイルジャマー）起動**⁵²、**自動修復キット使用**、**冷却装置使用**などが該当します。これらは緑色のチップで描かれ、使用すると一定クールダウンが発生するものもあります⁵³。同じ種類のオプションは効果中に重ねがけできない制約があるため、プログラム上は**使えなくても次へ進む**ように組むのがポイントとされています⁵⁴。オプション使用チップ自体は1出力で即座に次の処理へ移ります。なお、オプション装備は装備スロット番号でチップが分かれており、装備箇所ごとに対応する使用チップを置く必要があります。
- ・センサー系チップ（条件）**：前述した**各種条件分岐チップ**がこれに当たります。赤色のアンテナやレーダー型のアイコンで示され、2出力（Yes/No）を持つのが特徴です⁵⁵。種類は非常に多く、「機体探査」「障害物探査」「発射物探査」「状態判断」「動作判断」「ターゲット判断各種」「カウンタ判断」「時間判断」「ランダム分岐」など多岐にわたります^{56 57 58}。センサー系チップはいわばOKEの「目」や「耳」の役割で、これらを駆使して状況認識を行い、Yes側に適切な行動チップを繋ぐことで**知的な振る舞い**を引き出します。
- ・通信・フラグ系チップ（思考）**：主に内部処理や仲間との通信に使うチップです。**チャンネル（通信）チップ**は5色の信号をON/OFFでき、同じ部隊内で共通の色フラグとして利用します²⁵。例えば「青チャンネルON」チップをリーダー機が実行し、他の機体が「青チャンネルがONか？」条件で追従行動に入る、といった連携が可能です。**カウンタチップ**は上述のカウンタ変数を操作するもので、値をインクリメント/デクリメントしたり特定数値を代入できます²⁴。いずれも2出力（操作後に条件分岐可能）となっており、他の条件と組み合わせで使われます。例えばカウンタチップは「計算後、結果が0より大きいか」など自分で条件を設定でき、特定値に達した場合だけYesに繋ぐような使い方も考えられます（※実質的に条件付き算術演算）。
- ・制御系チップ（思考）**：プログラムの流れを調整するためのチップです。**ノップ（No Operation）**は何もしない空処理で、チップ間を繋ぐための単なる経路として使われます⁵⁹。**ウェイト（待機）**は一定時間思考を停止し、時間経過後に次のチップへ進みます⁴²。**ストップ（動作停止）**は現在実行中の動作を中断する命令で、例えば歩行中にこれを実行するとその場で足を止めます⁶⁰。これら制御チップはプログラムのタイミング調整や無限ループ防止、動作の微調整に用いられ、1出力（直通）または場合により2出力を持ちます。特にWaitは前述のようにレスポンスと安定性のバランスを取るのに重要です。

以上が主なチップカテゴリと種類です。**シリーズが進むにつれチップの種類や使い勝手は改良・拡張**されており、PSP版では初代に比べ便利なチップ（例：自動旋回や射撃しながら移動するためのモードなど）が増えています³⁶。例えば初代では難しかった「移動しながら攻撃」も、エクサでは前述の通過モードに加え「**ガンサイト射撃**」という便利なチップで簡単に実現できます⁶¹。他にもEZ（イージーザッピング）では初心者向けに**マクロチップ**が用意され、よく使う処理を塊で配置できるようになっています（例：「接近戦用回避パック」など）。こうした改良により、基本システムは不変でも徐々に**扱いやすさと表現力が向上**しています。

7. 条件分岐表現の自由度と設計上のポイント

カルネージハートのプログラミングは、一見シンプルなYES/NO分岐の組み合わせですが、その**表現の自由度**は非常に高いです。上級者になれば、チップの繋ぎ方次第で複雑な論理を実装でき、まさにif/elif/elseの入れ子構造や論理和・論理積の条件、さらには**ループ処理**まで再現できます。

複合条件 (AND/OR) の例は既に述べましたが、例えば「AかつBなら…」を実現するにはA条件チップのYes側にB条件チップを繋ぎ、BもYesのときだけ次の処理に進めばOKです⁹（AまたはBのいずれも成立しなければ進まない）。逆に「AまたはBなら…」を表現するには、並列経路で両方の条件をNo側に繋ぎ、両方Noだった場合だけ進まないようにします⁶²。このような**論理結合**はチップを使って自由に作れるため、複雑な状況判断も細かく定義できます。

階層的な条件分岐 (ネスト) も、チップ接続のレイアウトで視覚的に表現されます。カルネージハートのエディタ上では、条件チップを縦に繋げていくとインデントのように深い層になっていきます¹⁰。これはまさにプログラミング言語のネストと同様で、例えば「もしAなら、その中でさらにBをチェックし…」という処理が見た目に分かります。ネストが深くなりすぎるとプログラムを見失いやすくなる点もコードと同じなので、整理のために適宜NOPを入れてレイアウトしたり、メモリ節約のために似た機能はまとめて関数相当に作る（テンプレート化する）といった工夫も有効です。

ループ処理については、カルネージハートでも**意図的にループを構成**することができます。基本は自動でSTARTに戻るループですが、途中に**閉ループ**を作ることによって特定の条件を満たすまでその部分を繰り返す動きになります⁶³。例えば、「条件AがYesになるまで特定のアクションを繰り返す」というパターンは、条件AチップのNo側を上流のチップに戻すよう繋いでやれば実現します⁶³（Yesになったらループを抜けて先に進む）。この一方向ゲートのループは、条件Aが**いずれ必ずTrueになる場合**に有効で、例えば「タイマーが5秒経過するまでループ」や「残弾が無くなるまで連射を続けるループ」などが考えられます⁶⁴。実際、反応を早めるテクニックとして「一定時間だけ回避判定を高速で回す（ループさせる）」方法が有効な場合もあり、状況次第で使い分けます⁶⁵。

しかし注意すべきは、**無限ループの危険性**です。例えば上記の条件Aが永遠に満たされない場合（不可逆な条件がFalseのままなど）、プログラムはそのループから抜け出せなくなり、他の処理が一切実行されなくなります⁶⁶。これは人間のプログラミングと同様に致命的なので、ループを設計する際は**必ず抜け道を用意**することが重要です。抜ける条件が時間や弾切れといった将来的に必ず起こるイベントであれば安全ですが、そうでない場合は手動でループを解除する工夫（一定回数で諦める、外部信号で強制脱出する等）も検討します。また、カルネージハートでは通常メインループ（START⇄RETURN）は自動で回るため、意図的に閉ループを作らなくても十分なケースが多々あります。したがって無限ループに陥りやすい設計は避け、**基本はメインループの毎周期でこまめに状況判断**を行う方が安定したAIになりやすいです。

最後に、開発・設計上の補足ポイントを挙げます。カルネージハートのAI設計では「**いかに効率よく必要な判断を盛り込むか**」が肝心です。プログラムボードのサイズには限りがあるため、無駄なチップを省きながら必要な分岐を構成するパズルの要素があります。またCPU性能との兼ね合いで「全ての判断を毎フレーム行う」のは不可能なので、優先度の低い処理は間引いたり、時間判断チップで一定間隔ごとに実行するなど工夫すると良いでしょう。公式ガイドでも「毎ループ必須の機能はコード冒頭で大雑把に、その後徐々に具体的な処理へ絞り込む（広い検知から狭い検知へ）」ことが推奨されています⁶⁷。例えばまず大まかに敵方向を向く→次に細かく狙いを定める、といった順序です⁶⁷。さらに「プログラムの端（ボード外周）で終了させればループが速く回る」ことも指摘されています⁶⁸。これは、余計なノップで引き延ばさず、早めにRETURNに到達する経路を確保することで無駄な遅延を省けるという意味です。実際、熟練者はボード端のRETURNタイルをうまく活用し、不要なスペースを作らないよう配慮しています。

以上、オリジナルのカルネージハートシリーズ（初代～PSP版）におけるAIプログラム（OS設計）の仕様について、条件分岐チップの種類やセンサー変数、分岐構造、実行モデル、具体的なチップ例、カテゴリ分

類、そしてプログラム設計の自由度とポイントを網羅的に解説しました。公式マニュアルや攻略本にはチップ一覧や使用例が掲載されており、ユーザー有志のサイトやWikiにも**OS設計の指南**や**フローチャート例**が豊富に公開されています。実際の画面スクリーンショットも参考に、ぜひ様々なチップ組み合わせを試してみて、自分なりの最適なAIをデザインしてみてください。 3 69

Sources:

- Carnage Heart - Wikipedia 1 3 (ゲーム概要とプログラム方式)
- 『カルネージハートEXA 2chまとめwiki』 初心者向け講座 2 18 8 15 39 36 他 (ゲーム内チップ例と解説)
- CypressDahlia氏作成 “Carnage Heart” 英語攻略ガイド 6 70 63 9 他 (チップ一覧・論理構成例)
- GameFAQsフォーラム投稿 「Carnage Heart EXA chip discussion」 71 72 (チップ色別の例)
- カルネージハート公式攻略ガイド (ファミ通) より引用 73 34 (動作優先度とプログラム構成のTips)

1 3 Carnage Heart - Wikipedia

https://en.wikipedia.org/wiki/Carnage_Heart

2 8 14 15 16 18 36 37 38 39 40 41 53 54 61 65 初心者向け講座 - Carnage Heart EXA 2chまとめwiki - atwiki (アットウィキ)

<https://w.atwiki.jp/chex/pages/61.html>

4 5 71 72 chip discussion - Carnage Heart EXA

<https://gamefaqs.gamespot.com/boards/998172-carnage-heart-exa/66813658>

6 7 9 10 11 24 25 29 30 34 35 42 43 44 45 46 47 48 49 50 51 52 55 59 60 62 63 64 66 67 68 69 70 73 Carnage Heart - Guide and Walkthrough - PlayStation - By CypressDahlia - GameFAQs

<https://gamefaqs.gamespot.com/ps/196873-carnage-heart/faqs/79570>

12 13 17 19 20 21 22 23 26 27 28 56 57 58 条件分岐 - Carnage Heart EXA 2chまとめwiki - atwiki (アットウィキ)

<https://w.atwiki.jp/chex/pages/187.html>

31 32 33 CPU - Carnage Heart EXA 2chまとめwiki - atwiki (アットウィキ)

<https://w.atwiki.jp/chex/pages/63.html>